

STM32 DC Motor Speed Control using PI Controller, System Identification and Kalman Filter

1. 프로젝트 개요

1.1 프로젝트 목적

DC 모터는 이동 로봇, AGV(Automated Guided Vehicle), 산업용 자동화 장비 등 다양한 임베디드 시스템에서 가장 널리 사용되는 구동 장치 중 하나이다. 이러한 시스템에서는 목표 속도를 정확하게 유지하는 것이 중요하며, 외란이나 센서 노이즈가 존재하는 환경에서도 안정적인 속도 제어 성능이 요구된다.

본 프로젝트에서는 **STM32F746NG**를 이용하여 DC 기어모터의 정밀 속도 제어 시스템을 구현하였다. 모터의 회전 속도는 Incremental Encoder를 이용하여 측정하였으며, TIM1에서 생성한 PWM 신호를 이용하여 모터의 입력 전압을 제어하였다.

또한 실제 모터의 Step Response를 측정하여 **System Identification**을 수행하고, 이를 기반으로 모터를 1차 선형 시스템으로 모델링하였다. 추정된 전달함수를 이용하여 MATLAB SISOTOOL에서 연속시간 PI Controller를 설계하였으며, Tustin(Bilinear) 변환을 통해 디지털 제어기로 변환하여 STM32에 구현하였다.

제어기 구현 과정에서는 디지털 제어기의 샘플링 주파수와 제어기 대역폭(Bandwidth)의 관계를 고려하여 제어기를 재설계하였으며, 엔코더의 양자화 오차와 측정 노이즈를 감소시키기 위해 1차 Kalman Filter를 적용하였다.

최종적으로 목표 속도 **0.2 m/s**를 안정적으로 추종하는 페루프 속도 제어 시스템을 구현하고, MATLAB과 STM32 실험 결과를 비교하여 제어 성능을 검증하였다.

1.2 프로젝트 목표

본 프로젝트의 목표는 다음과 같다.

- 목표 속도 0.2 m/s 추종
- PWM 기반 DC 모터 속도 제어
- Incremental Encoder 기반 실시간 속도 측정

- System Identification을 이용한 시스템 모델링
- MATLAB SISOTOOL을 이용한 PI Controller 설계
- Tustin 변환을 이용한 디지털 PI Controller 구현
- Kalman Filter를 이용한 속도 추정
- MATLAB을 이용한 제어 성능 분석 및 검증

항목	목표치	상세 설명
목표 크루즈 속도	0.2 m/s	65 mm 휠 기준 약 60 RPM. 엔코더 신호 신뢰성과 제어 응답성을 고려하여 설정
평균 허용 오차	±2% 이내	설계 목표: 정상상태 평균 속도 기준
정착 시간	1초 이내	정지 상태에서 목표 속도 도달 시간
최대 오버슈트	10% 이내	목표 속도를 초과하는 응답 억제

1.3 개발 환경

Item	Description
MCU	STM32F746NG
IDE	STM32CubeIDE
Language	C
Analysis Tool	MATLAB (System Identification Toolbox, SISOTOOL)
Control Method	PI Controller
State Estimator	Kalman Filter
Sampling Time	10 ms

1.4 프로젝트 진행 절차

본 프로젝트는 다음과 같은 절차를 통해 진행하였다.

DC Motor 구동

↓

Encoder 기반 속도 측정

↓

CSV 데이터 저장

↓

System Identification

↓

1차 전달함수 추정

↓

MATLAB SISOT00L

↓

Continuous PI Controller 설계

↓

Tustin(Bilinear) 변환

↓

STM32 구현

↓

샘플링 주파수를 고려하여 PI Controller의 Bandwidth를 재조정

↓

Kalman Filter 적용

↓

최종 성능 검증

2. 시스템 구성

2.1 시스템 개요

본 시스템은 DC 기어모터의 속도를 일정하게 유지하기 위한 **폐루프(Closed-Loop) 속도 제어 시스템**으로 구성하였다.

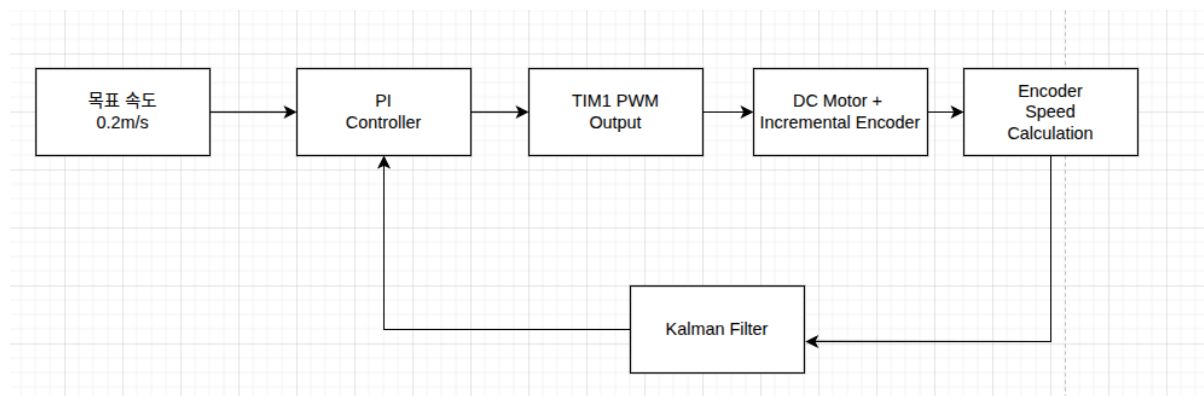
목표 속도(Setpoint)와 현재 측정된 속도의 오차(Error)를 계산하여 PI Controller에서 제어 출력을 생성하고, TIM1에서 PWM 신호를 발생시켜 모터를 구동한다.

모터의 회전 속도는 Incremental Encoder를 이용하여 측정하며, STM32의 TIM3 Encoder Interface를 통해 엔코더 카운트를 읽는다.

엔코더로부터 계산한 속도는 양자화 오차와 측정 노이즈를 포함하므로 Kalman Filter를 이용하여 속도를 추정하였다.

최종적으로 Kalman Filter의 출력은 PI Controller의 피드백 값으로 사용되며, 이러한 과정이 **10 ms 주기**로 반복 수행된다.

2.2 시스템 블록도



2.3 하드웨어 구성

구성 요소	역할
STM32F746NG	PWM 생성, Encoder 계측 및 PI 제어 수행
JGA25-371 DC Geared Motor	제어 대상(Plant)
Incremental Encoder	모터 회전량 측정
TB6612 Motor Driver	PWM 신호를 이용한 모터 구동
Motor Power	18650 Li-ion Battery ×2 (2S)
MATLAB	System Identification 및 제어기 설계

2.4 DC Motor Specifications

사용한 제어 대상은 JGA25-371 DC Geared Motor (Gear Ratio 1:34)이며, 주요 사양은 Table 2-2와 같다.

Table 2-2. DC Motor Specifications

Item	Specification
Operating Voltage	6 ~ 24 V
Nominal Voltage	12 V

Item	Specification
Gear Ratio	1 : 34
Free-run Speed (12 V)	126 RPM
Free-run Current (12 V)	46 mA
Stall Current (12 V)	1 A
Stall Torque (12 V)	4.2 kg·cm
Reducer Size	21 mm
Weight	99 g

2.5 제어 주기

본 프로젝트에서는 10 ms(100 Hz)의 제어 주기를 사용하였다.

매 제어 주기마다 다음 순서로 제어가 수행된다.

1. TIM3 Encoder Counter 읽기
2. Encoder Tick 계산
3. 선속도(m/s) 계산
4. Kalman Filter 수행
5. PI Controller 계산
6. TIM1 PWM Duty 갱신

100 Hz의 제어 주기는 엔코더 분해능, MCU 연산 시간 및 제어기의 응답성을 고려하여 선정하였다.

3. PWM 설계

3.1 PWM 제어 방식

본 프로젝트에서는 DC 모터의 속도를 제어하기 위해 PWM(Pulse Width Modulation) 방식을 사용하였다.

PWM은 일정한 주기의 디지털 신호에서 HIGH 상태의 비율(Duty Cycle)을 조절하여 모터에 인가되는 평균 전압을 제어하는 방식이다. 평균 전압이 증가할수록 모터의 회전 속도가 증가하며, 반대로 평균 전압이 감소하면 회전 속도가 감소한다.

PWM 방식은 선형 전압 제어 방식에 비해 전력 손실이 적고 효율이 높기 때문에 임베디드 시스템에서 가장 널리 사용되는 모터 제어 방법이다.

본 프로젝트에서는 STM32F746NG의 TIM1을 이용하여 PWM 신호를 생성하고, PI Controller의 출력값을 PWM Duty Cycle로 변환하여 모터의 속도를 제어하였다.

3.2 TIM1 선택

PWM 생성에는 STM32F746NG의 **TIM1**을 사용하였다.

TIM1은 STM32의 Advanced Timer로, 일반 Timer보다 다양한 PWM 기능을 제공한다. 특히 다음 기능을 지원한다.

- Center-Aligned PWM
- Complementary PWM
- Dead Time Generator
- Break Input
- Main Output Enable(MOE)

본 프로젝트에서는 Center-Aligned PWM을 사용하기 위해 TIM1을 선택하였다.

3.3 TIM1 설정

TIM1의 설정은 다음과 같다.

Parameter	Value
Prescaler	0
Counter Mode	Center-Aligned Mode 1
Period (ARR)	5400
Clock Division	DIV1
Repetition Counter	0
Auto Reload Preload	Enable
PWM Mode	PWM Mode 1
Output Polarity	Active High

PWM 출력은 Channel 1을 사용하였다.

3.4 Center-Aligned PWM

본 프로젝트에서는 일반적인 Edge-Aligned PWM이 아닌 **Center-Aligned PWM Mode 1**을 사용하였다.

Center-Aligned PWM에서는 Timer Counter가 단순히 증가만 하는 것이 아니라,

```
0 → 1 → 2 → ... → ARR
      ↓
ARR ← ... ← 2 ← 1 ← 0
```

와 같이 증가(Up-counting)와 감소(Down-counting)를 반복한다.

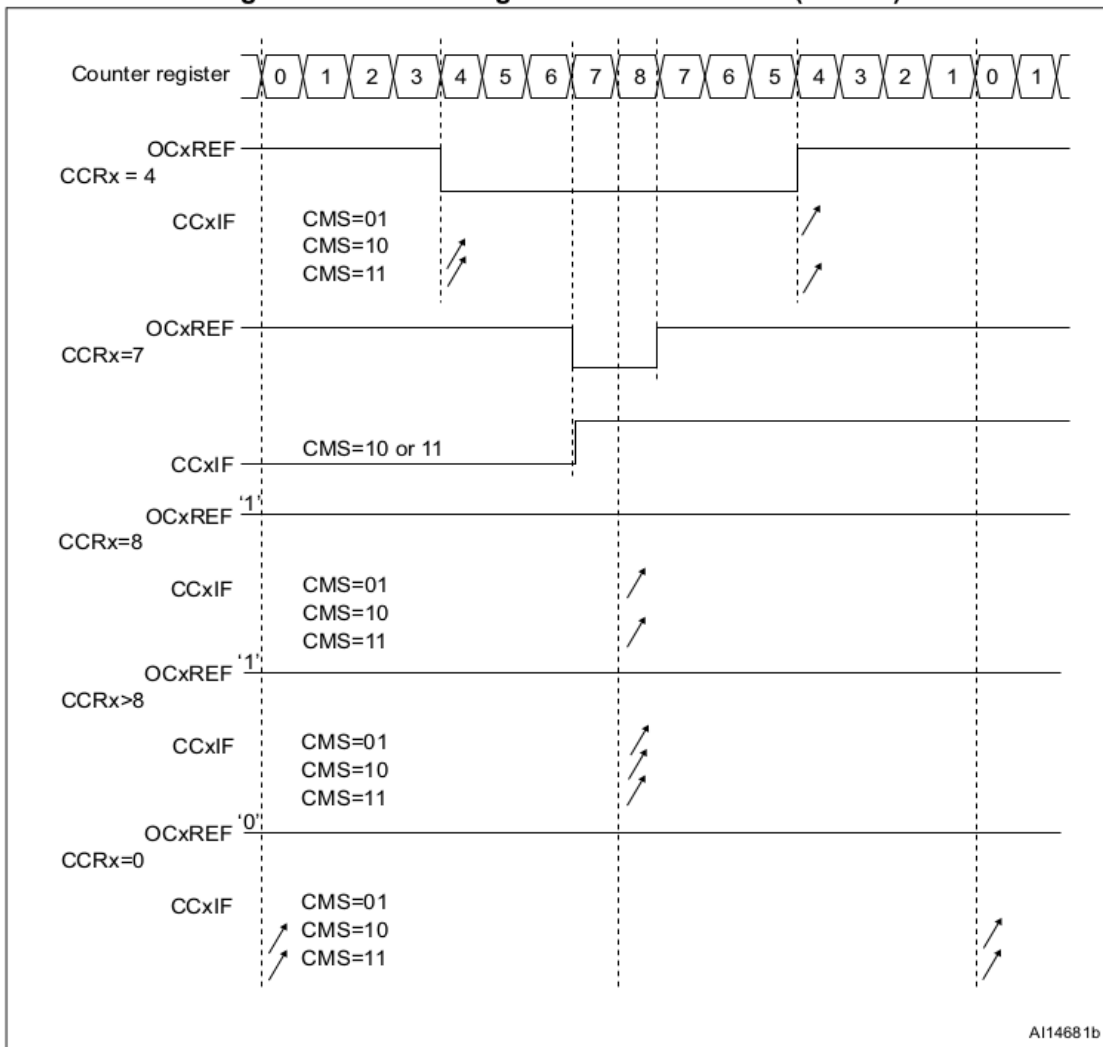
따라서 하나의 PWM 주기는 Up-counting과 Down-counting을 모두 포함한다.

Center-Aligned PWM은 Edge-Aligned PWM에 비해 PWM 스위칭 시점이 주기의 중심을 기준으로 대칭적으로 발생하므로 다음과 같은 장점을 가진다.

- 출력 파형의 대칭성 향상
- 스위칭 노이즈 감소
- EMI(Electromagnetic Interference) 감소
- 모터 구동 안정성 향상

이러한 이유로 본 프로젝트에서는 TIM1의 Center-Aligned Mode 1을 사용하였다.

Figure 226. Center-aligned PWM waveforms (ARR=8)



3.5 PWM Mode 1

TIM1은 PWM Mode 1로 설정하였다.

PWM Mode 1에서는 Counter(CNT)와 Compare Register(CCR)의 값을 비교하여 출력을 결정한다.

Active High 설정에서는

$CNT < CCR$

인 동안 PWM 출력은 HIGH 상태를 유지한다.

반대로

$CNT \geq CCR$

에서는 LOW 상태가 된다.

따라서 CCR 값이 증가할수록 HIGH 구간이 길어지며 PWM Duty Cycle이 증가한다.

본 프로젝트에서는 PI Controller의 출력값을 CCR에 기록하여 PWM Duty를 실시간으로 변경하였다.

3.6 PWM Duty 계산

PWM Duty Cycle은 다음 식으로 계산된다.

$$Duty(\%) = \frac{CCR}{ARR} \times 100$$

ARR 값은 ARR=5400 으로 설정하였다.

따라서

CCR	Duty
0	0%
2700	50%
5400	100%

가 된다.

PI Controller의 출력은 항상

$$0 \leq CCR \leq 5400$$

범위로 제한하였다.

이를 통해 Timer의 유효 범위를 벗어나는 PWM 출력이 발생하지 않도록 하였다.

3.7 PWM 출력 갱신

PI Controller에서 계산한 제어 출력은 TIM1의 Compare Register(CCR)에 기록하였다.

```
__HAL_TIM_SET_COMPARE(&htim1,  
TIM_CHANNEL_1,(uint32_t)u_out);
```

여기서 `u_out` 은 PI Controller의 출력값이며, TIM1 Channel 1의 CCR 값으로 사용된다.

제어기는 매 10 ms마다 현재 속도 오차를 계산하고 PWM Duty를 갱신하여 모터 속도를 제어하였다.

3.8 PWM 출력 시작

PWM 출력은 시스템 초기화 이후 다음 함수를 이용하여 시작하였다.

```
HAL_TIM_PWM_Start(&htim1,TIM_CHANNEL_1);
```

TIM1은 Advanced Timer이므로 내부적으로 Main Output Enable(MOE)이 활성화되어야 실제 PWM 출력이 가능하다.

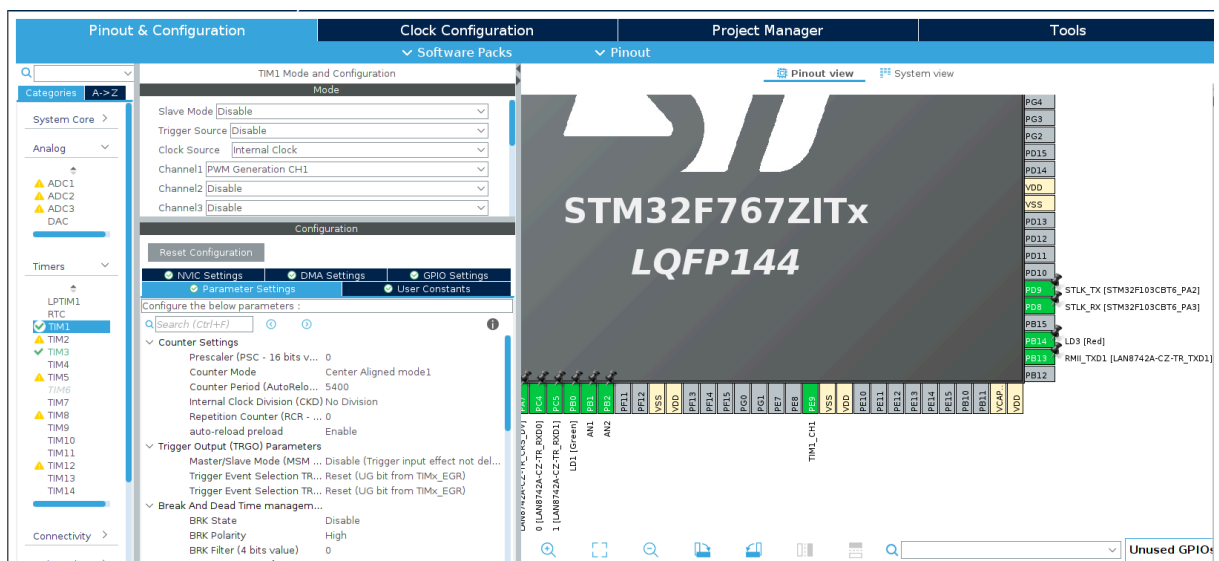
STM32 HAL에서는 `HAL_TIM_PWM_Start()` 함수를 이용하여 PWM 출력을 시작하며, 필요한 출력 설정을 함께 수행한다.

3.9 요약

본 프로젝트의 PWM 설계는 다음과 같이 구성하였다.

- TIM1 Advanced Timer 사용
- Center-Aligned Mode 1 적용
- PWM Mode 1 사용
- Active High 출력
- ARR = 5400
- PI Controller 출력 → CCR 값
- CCR 범위 제한(0 ~ 5400)
- 10 ms마다 PWM Duty 갱신

이를 통해 PI Controller의 출력이 PWM Duty Cycle에 직접 반영되도록 구현하였으며, 모터의 평균 입력 전압을 제어하여 목표 속도를 안정적으로 추종하도록 구성하였다.



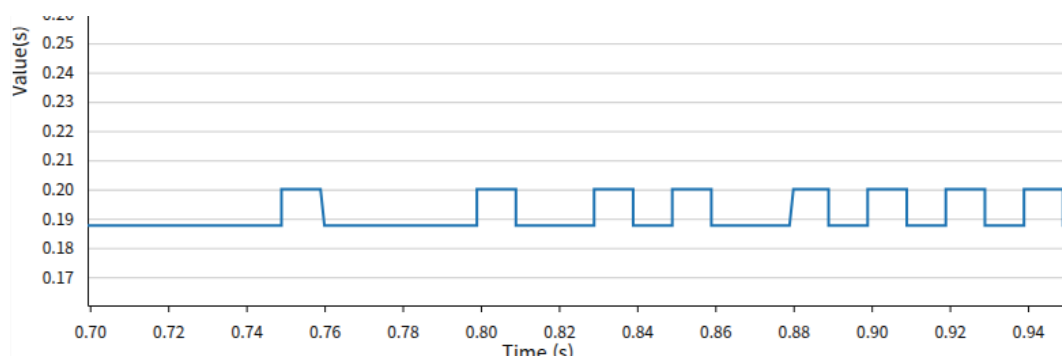
4. Encoder 기반 속도 측정

4.1 Incremental Encoder

본 프로젝트에서는 모터의 회전 속도를 측정하기 위해 JGA25-371 DC Geared Motor에 내장된 Incremental Encoder를 사용하였다.

Encoder는 회전량을 펄스 형태로 출력하며, STM32F746NG의 TIM3 Encoder Interface 기능을 이용하여 하드웨어적으로 계수하였다.

Encoder Interface Mode를 사용하면 A상과 B상의 위상차를 이용하여 회전 방향과 회전량을 자동으로 계산할 수 있으므로 CPU의 부하를 줄일 수 있다.



4.2 TIM3 Encoder Interface 설정

Encoder 신호 계측에는 TIM3를 사용하였다.

TIM3는 Encoder Interface Mode(TI12)로 설정하였으며, Quadrature Encoder를 4채배(X4)하여 계수하도록 구성하였다.

Table 4-1. TIM3 Encoder Configuration

Parameter	Value
Timer	TIM3
Encoder Mode	TI12
Prescaler	0
Counter Period	65535
Counter Mode	Up Counter

프로그램 시작 시 다음 함수를 이용하여 Encoder Interface를 활성화하였다.

```
HAL_TIM_Encoder_Start(&htim3,TIM_CHANNEL_ALL);
```

4.3 Encoder 분해능 계산

사용한 Encoder의 기본 분해능은 12 CPR(Counts Per Revolution)이다.

TIM3 Encoder Interface는 A상과 B상의 상승 및 하강 에지를 모두 계수하는 X4 모드를 사용하므로 실제 분해능은 다음과 같다.

$$12 \times 4 = 48 \text{ Counts/Motor Revolution}$$

모터에는 34:1 감속기가 장착되어 있으므로 출력축 기준 분해능은

$$48 \times 34 = 1632 \text{ Counts/Output Revolution 이 된다.}$$

따라서 출력축이 한 바퀴 회전할 때 TIM3 Counter는 1632 Count 증가한다.

4.4 선속도 계산

바퀴의 직경은

D=65mm 이다.

따라서 바퀴의 둘레는

$$C=\pi D$$

$$C=\pi \times 0.065 = 0.2041\text{m 이다.}$$

Incremental Encoder의 출력축 기준 분해능은 **1632 Counts/Rev**이므로, 한 제어 주기 동안 측정된 Encoder Count를 이용하여 선속도는 다음과 같이 계산하였다.

$$v = \frac{\Delta Count}{1632} \times 0.2041 \times 100$$

여기서

- $\Delta Count$: 제어 주기(10 ms) 동안 증가한 Encoder Count
- **1632** : 출력축 기준 Encoder 분해능 (Counts/Rev)
- **0.2041 m** : 바퀴의 둘레
- **100** : 10 ms에서 1초 단위로 변환하기 위한 계수

을 의미한다.

STM32에서는 다음 코드로 구현하였다.

```
raw_v = ((float)delta_tick / 1632.0f) * 0.2041f * 100.0f;
```

4.5 속도 측정의 한계

Encoder 기반 속도 측정은 구현이 간단하고 계산량이 적다는 장점이 있다.

그러나 속도는 일정한 제어 주기 동안 증가한 Encoder Count를 이용하여 계산하므로, 측정값은 Count의 정수 단위 변화에 영향을 받는다.

본 프로젝트에서는 출력축 기준 분해능이 1632 Counts/Rev이고 제어 주기는 10 ms이다. 목표 속도인 0.2 m/s 부근에서는 한 제어 주기 동안 약 16 Tick 정도가 측정된다.

따라서 실제 모터 속도가 거의 일정하더라도 측정되는 Count가 16 Tick과 17 Tick 사이에서 변화할 수 있으며, 이에 따라 계산된 속도 역시 일정한 간격으로 변동하게 된다.

이러한 현상은 실제 모터의 속도 변화가 아니라 Encoder의 분해능과 샘플링 방식에서 발생하는 양자화 오차(Quantization Error)에 의한 것이다.

따라서 본 프로젝트에서는 이러한 측정 노이즈를 줄이기 위해 Kalman Filter를 적용하였다.

4.6 실제 측정 결과

STM32CubeMonitor를 이용하여 속도를 측정한 결과, 평균적으로는 목표 속도인 0.2m/s를 유지하였지만 순간적으로 속도가 일정 간격으로 변하는 현상을 확인하였다.

이러한 속도 변동은 실제 모터의 속도가 크게 변한 것이 아니라, Encoder Count가 정수 단위로 증가하기 때문에 발생하는 양자화 오차에 의한 영향으로 판단하였다.

또한 측정 노이즈가 포함된 속도를 그대로 PI Controller의 입력으로 사용할 경우 PWM 출력이 불필요하게 변동할 가능성이 있었다.

따라서 다음 장에서는 이러한 측정 노이즈를 줄이기 위해 Kalman Filter를 적용하였다.

5. 시스템 모델링 (System Identification)

5.1 시스템 모델링 목적

제어기를 설계하기 위해서는 제어 대상(Plant)의 동특성을 먼저 파악해야 한다. 일반적으로 DC 모터는 전기적 특성, 기계적 특성, 감속기, 마찰 등의 영향으로 복잡한 동적 특성을 가지므로 이를 정확한 수학적 모델로 표현하기는 어렵다.

본 프로젝트에서는 이론적인 모델을 유도하는 대신, 실제 모터에 Step 입력을 인가하여 얻은 응답 데이터를 이용한 **System Identification** 기법을 적용하였다.

실험을 통해 얻은 속도 응답 데이터를 MATLAB에서 분석하여 실제 시스템을 가장 잘 표현하는 1차 선형 전달함수를 추정하고, 이를 PI Controller 설계의 기준 모델로 사용하였다.

5.2 Step Response 측정

모터의 동특성을 측정하기 위해 일정한 PWM 입력을 인가하고 속도 응답을 측정하였다.

PWM 출력은 TIM1의 Compare Register(CCR)를 이용하여 일정한 값으로 유지하였으며, 본 실험에서는 **CCR = 3000**을 사용하였다.

속도는 Incremental Encoder를 이용하여 측정하였으며, STM32CubeMonitor를 이용하여 실시간으로 데이터를 수집하였다.

취득한 데이터는 CSV 형식으로 저장한 후 MATLAB에서 분석하였다.

실험 과정은 다음과 같다.

1. TIM1 CCR = 3000으로 설정
2. Step 입력 인가
3. Encoder를 이용한 속도 측정
4. STM32CubeMonitor를 이용하여 CSV 저장
5. MATLAB System Identification 수행

5.3 MATLAB System Identification

STM32CubeMonitor를 이용하여 취득한 Step Response 데이터를 CSV 형식으로 저장한 후, MATLAB System Identification Toolbox를 이용하여 입력(PWM)과 출력(속도) 데이터를 분석하고, 모터 시스템을 1차 선형 전달함수로 모델링하였다.

추정된 전달함수는 실제 모터의 동특성을 근사적으로 표현하며, 이후 MATLAB SISOTOOL에서 PI Controller를 설계하기 위한 기준 모델로 사용하였다.

추정된 전달함수는 다음과 같다.

$$G(s) = \frac{0.00507}{0.135s+1}$$

여기서

- **DC Gain** : $K=0.00507$
- **Time Constant** : $\tau=0.135s$

5.4 전달함수 해석

추정된 전달함수는 다음과 같은 의미를 가진다.

$$G(s) = \frac{\text{Velocity}(s)}{\text{PWM}(s)}$$

즉, 입력은 PWM,

출력은 모터의 선속도이다.

DC Gain은 PWM 입력이 일정하게 유지될 때 정상상태 속도가 얼마나 증가하는지를 나타내며,

Time Constant는 시스템 응답 속도를 나타낸다.

1차 시스템에서는 출력이 최종값의 약 **63.2%**에 도달하는 시간을 Time Constant라고 정의한다.

5.5 선형 근사(Linear Approximation)

실제 DC 모터 시스템은 다음과 같은 비선형 요소를 포함한다.

- Motor Driver(TB6612)
- 브러시 마찰(Friction)
- 기어박스(Backlash)
- Dead Zone
- 전원 전압 변화
- 기계적 손실

따라서 실제 시스템은 하나의 전달함수로 완벽하게 표현할 수 없다.

그러나 제어기 설계를 위해서는 특정 동작점 주변에서 시스템을 선형 시스템으로 근사하는 것이 일반적이다.

본 프로젝트에서는 **CCR = 3000** 부근에서 측정된 응답을 기준으로 시스템을 1차 선형 시스템으로 근사하였다.

5.6 모델링의 활용

추정된 전달함수는 이후 MATLAB SISOTOOL에서 PI Controller를 설계하는 데 사용하였다.

즉,

Step Response 측정

↓

System Identification

↓

전달함수 추정

↓

SISOTOOL

↓

PI Controller 설계

의 순서로 제어기를 설계하였다.

이를 통해 실제 시스템의 특성을 반영한 제어기를 설계할 수 있었으며, 이후 Tustin(Bilinear) 변환을 이용하여 STM32에 구현하였다.

6. PI Controller 설계

6.1 PI Controller 선정

5장에서 추정한 1차 전달함수를 기반으로 속도 제어기를 설계하였다.

DC 모터의 속도 제어에서는 정상상태 오차를 최소화하는 것이 중요하다. 비례 (Proportional) 제어만 사용할 경우 외란이나 부하 변화에 의해 정상상태 오차가 발생할 수 있으므로, 적분(Integral) 동작을 포함하는 PI(Proportional-Integral) Controller를 사용하였다.

연속시간 PI Controller는 다음과 같이 표현된다.

$$C(s) = K_p + \frac{K_i}{s}$$

여기서

- K_p : 비례 게인(Proportional Gain)
- K_i : 적분 게인(Integral Gain)

을 의미한다.

6.2 MATLAB SISOTOOL을 이용한 PI 설계

추정한 전달함수를 MATLAB SISOTOOL에 적용하여 PI Controller를 설계하였다.

SISOTOOL에서는 Bode Plot, Root Locus 및 Step Response를 이용하여 시스템의 안정성과 응답 특성을 분석하면서 PI Controller를 설계하였다.

설계 과정에서는 다음 항목을 중점적으로 확인하였다.

- Phase Margin
- Root Locus
- Step Response
- Settling Time
- Overshoot

Figure 6-1. PI Controller Design using MATLAB SISOTOOL

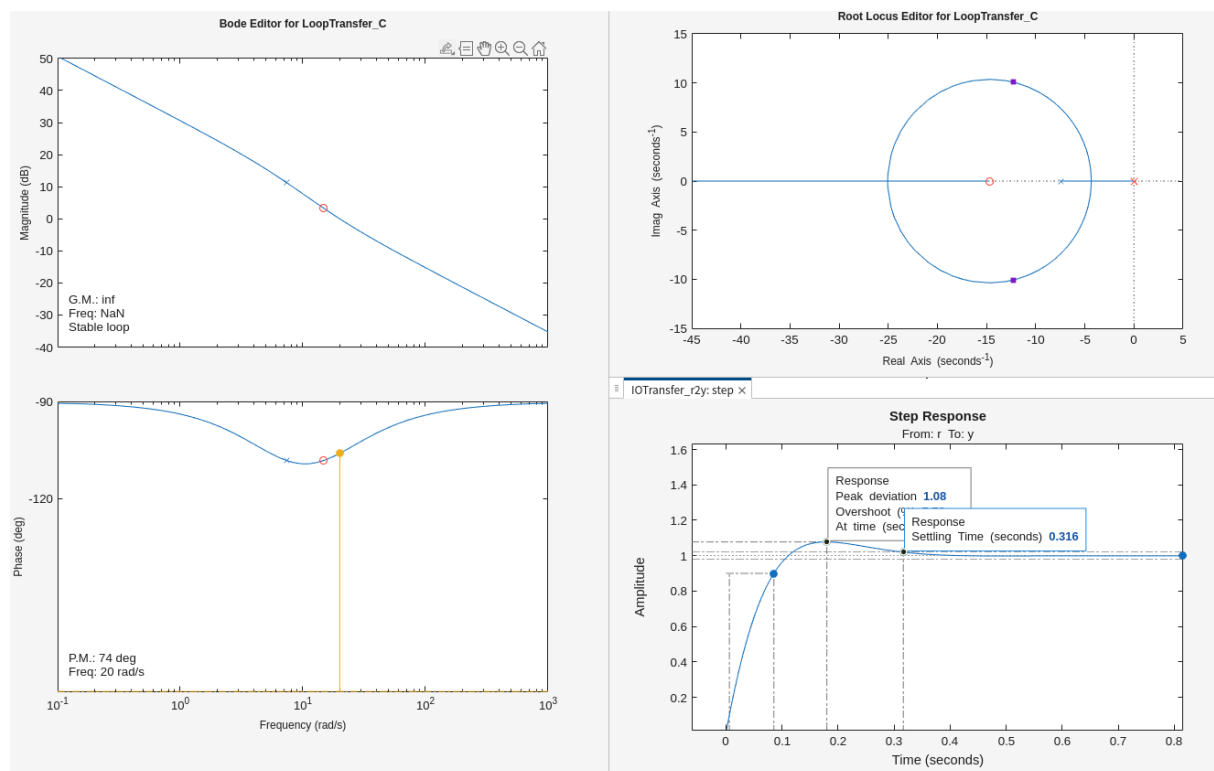


Figure 6-1 MATLAB SISOTOOL을 이용하여 PI Controller를 설계하였다. Bode Plot을 통해 주파수 응답과 Phase Margin을 확인하였으며, Root Locus를 이용하여 폐루프 극점의 위치를 분석하였다. 또한 Step Response를 통해 정착시간과 오버슈트를 확인하였다.

6.3 설계 결과

SISOTOOL에서 설계한 PI Controller는 다음과 같은 응답 특성을 나타내었다.

항목	결과
Phase Margin	약 74°
Gain Margin	Infinite
Settling Time	약 0.316 s
Overshoot	약 8 %

설계 결과 Phase Margin이 약 74°로 충분한 안정 여유를 확보하였으며, Step Response에서도 약 0.316초의 정착시간과 약 8%의 오버슈트를 나타내어 목표 응답 특성을 만족하는 것을 확인하였다.

6.4 디지털 구현

설계한 PI Controller는 연속시간 제어기이므로 STM32에서 사용할 수 없다.

따라서 Tustin(Bilinear) 변환을 이용하여 디지털 PI Controller로 변환하였다.

Tustin 변환은 다음 식을 사용하였다.

$$s = \frac{2}{T_s} \frac{z-1}{z+1}$$

여기서

- $T_s = 10\text{ms}$ 이다.

변환된 PI Controller는 Difference Equation 형태로 구현하였다.

6.5 초기 구현 결과

Tustin 변환을 적용한 PI Controller를 STM32에 구현하여 모터를 구동한 결과, 목표 속도를 추종하기는 하였지만 연속시간 시뮬레이션에서 예상한 응답과는 차이가 발생하였다.

초기에는 PI Gain의 문제로 판단하였으나, 응답 특성을 분석하는 과정에서 디지털 제어기의 샘플링 주파수와 제어기 대역폭(Bandwidth)의 관계를 다시 검토하였다.

6.6 샘플링 주파수와 대역폭 검토

MATLAB SISOTOOL에서 설계한 제어기의 대역폭이 실제 STM32의 샘플링 주파수에 비해 상대적으로 높게 설정되어 있을 경우, 연속시간에서 설계한 응답 특성이 디지털 구현에서 그대로 유지되지 않을 수 있다.

초기 설계에서는 이러한 영향을 충분히 고려하지 못하여 실제 STM32에서 응답 특성이 기대와 다르게 나타났다.

이에 따라 SISOTOOL에서 제어기의 주파수 대역폭을 조정한 후 다시 PI Controller를 설계하였으며, 이후 Tustin 변환을 수행하여 STM32에 재적용하였다.

이 과정을 통해 실제 시스템에서도 연속시간 시뮬레이션과 유사한 응답 특성을 얻을 수 있었다.

6.7 PI Gain 미세 조정

재설계한 PI Controller를 STM32에 적용한 결과 목표 속도를 안정적으로 추종하였으나, 실제 응답에서는 목표 속도까지 도달하는 시간이 다소 길게 나타났다.

이에 따라 실제 모터의 응답을 기준으로 Rise Time을 개선하기 위해 PI Gain을 약 1.5배 증가시켜 응답 속도를 향상시켰다.

게인 조정 후 목표 속도에 보다 빠르게 도달하는 것을 확인하였으며, 오버슈트 또한 허용 가능한 수준으로 유지되었다.

Figure 6-2. Velocity Response after PI Gain Adjustment

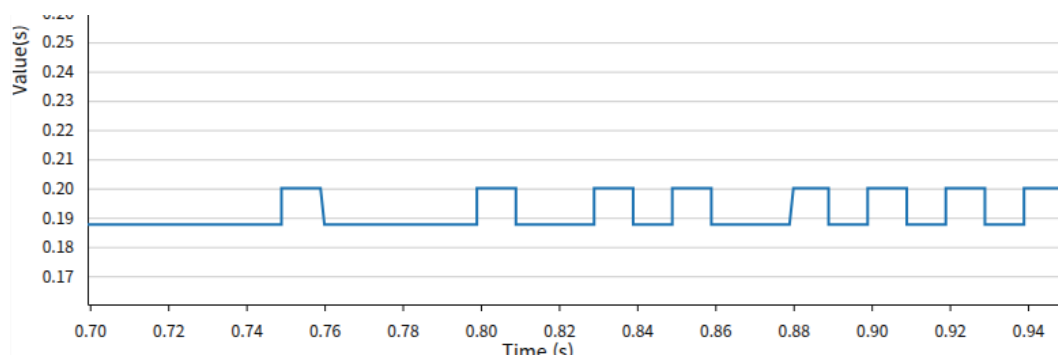


Figure 6-2 PI Gain 조정 후 정상상태 속도 응답이다. 목표 속도 0.2 m/s 부근을 평균적으로 유지하였으며, 엔코더 Tick 기반 속도 계산으로 인해 양자화 오차에 의한 작은 속도 변동이 발생하였다.

6.8 STM32 구현

최종적으로 설계한 PI Controller는 STM32에서 Incremental PI 형태로 구현하였다.

```
float e = setpoint - current_v;  
  
float u_out = u_prev
```

```

        + 1.2f * (491.16f * e)
        - (423.87f * e_prev);

    if (u_out > 5400.0f) u_out = 5400.0f;
    if (u_out < 0.0f)    u_out = 0.0f;

    __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, (uint32_t)u_out);

    u_prev = u_out;
    e_prev = e;

```

PI Controller에서 계산한 출력은 TIM1의 Compare Register(CCR)에 기록하여 PWM Duty Cycle을 실시간으로 변경하였다.

또한 PWM 출력이 Timer의 범위를 초과하지 않도록 0~5400 범위에서 Saturation을 적용하였다.

7. Kalman Filter 설계

7.1 Kalman Filter 적용 목적

PI Controller를 적용한 결과 목표 속도 0.2 m/s를 평균적으로 추종하는 것을 확인하였다.

그러나 엔코더 기반 속도 측정은 10 ms 주기로 Encoder Tick의 차이를 이용하여 속도를 계산하기 때문에, 정상상태에서도 속도 측정값이 일정한 간격으로 변화하는 현상이 발생하였다.

이는 실제 모터의 속도가 크게 변한 것이 아니라, 엔코더 분해능과 샘플링 주기에 의해 발생하는 **양자화 오차(Quantization Error)** 및 측정 노이즈에 의한 영향이다.

이러한 속도 측정값을 PI Controller의 피드백으로 그대로 사용할 경우 PWM 출력이 불필요하게 변동할 수 있으므로, 보다 안정적인 속도 추정값을 얻기 위해 Kalman Filter를 적용하였다.

7.2 Kalman Filter 선정

본 프로젝트의 상태(State)는 모터의 속도 하나만을 사용하였다.

또한 입력은 속도이며 출력 역시 속도이므로 시스템을 1차 선형 시스템으로 가정하였다.

상태방정식은 다음과 같이 정의하였다.

$$x_{k+1} = x_k + w_k$$

측정방정식은 다음과 같다.

$$z_k = x_k + v_k$$

여기서

- x_k : 실제 속도
- z_k : 엔코더 측정 속도
- w_k : Process Noise
- v_k : Measurement Noise

를 의미한다.

초기 조건은 모터가 정지 상태에서 시작하므로

$$x_0 = 0$$

으로 설정하였다.

7.3 Kalman Filter 설계

Kalman Filter는 Prediction 단계와 Correction 단계를 반복 수행하도록 설계하였다.

Prediction 단계에서는 오차 공분산을 예측하였다.

$$P = P + Q$$

이후 Kalman Gain을 계산하였다.

$$K = \frac{P}{P + R}$$

그리고 측정값을 이용하여 속도를 추정하였다.

$$\hat{x} = \hat{x} + K(z - \hat{x})$$

마지막으로 오차 공분산을 갱신하였다.

$$P = (1 - K)P$$

본 프로젝트에서는 다음 초기값을 사용하였다.

Parameter	Value
Q	0.00001
P	1
Initial Velocity	0

7.4 R 값 선정

Measurement Noise 공분산 RRR의 영향을 확인하기 위해 여러 값을 적용하여 비교하였다.

먼저

$R=0.01$

을 적용하였다.

Figure 7-1. Kalman Filter Result ($R = 0.01$)

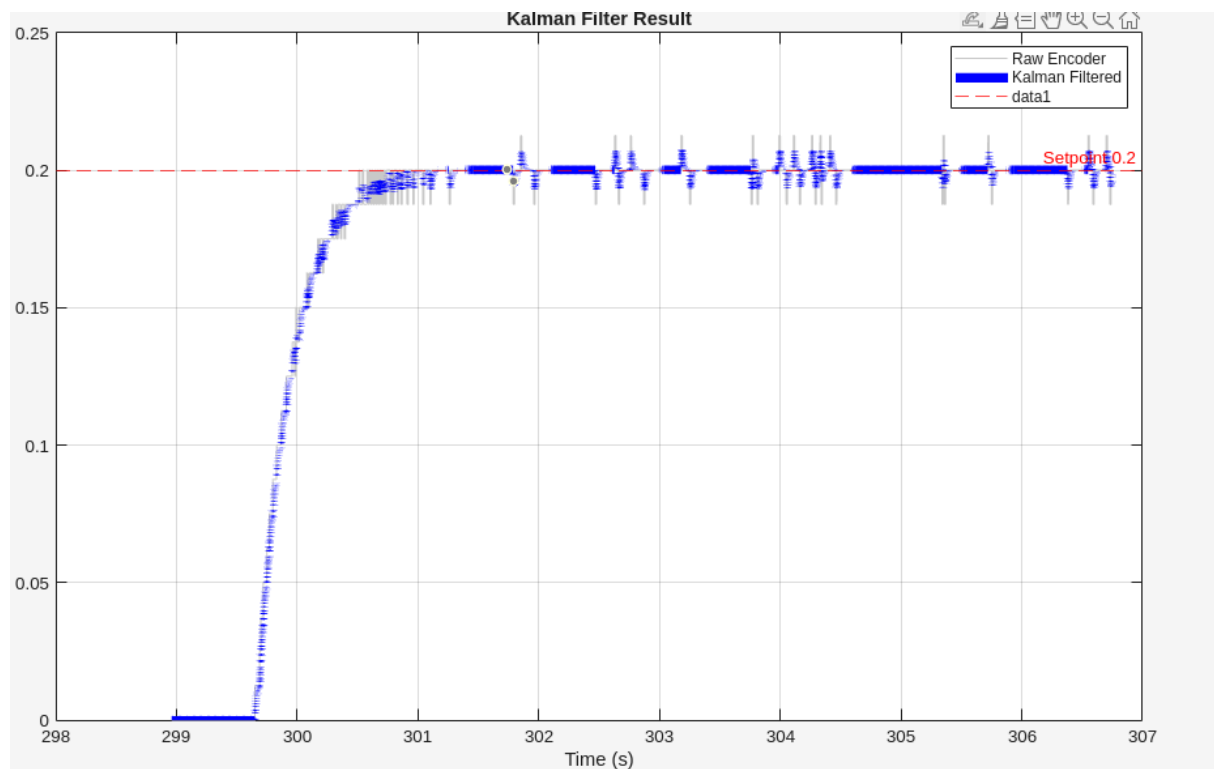


Figure 7-1 $R=0.01$ 을 적용한 결과이다. Raw Encoder 속도에 포함된 순간적인 노이즈는 일부 감소하였으나, 정상상태에서는 여전히 측정값의 변동이 비교적 크게 나타났다.

이후 $R=0.1$ 로 증가시켜 다시 실험하였다.

Figure 7-2. Kalman Filter Result ($R = 0.1$)

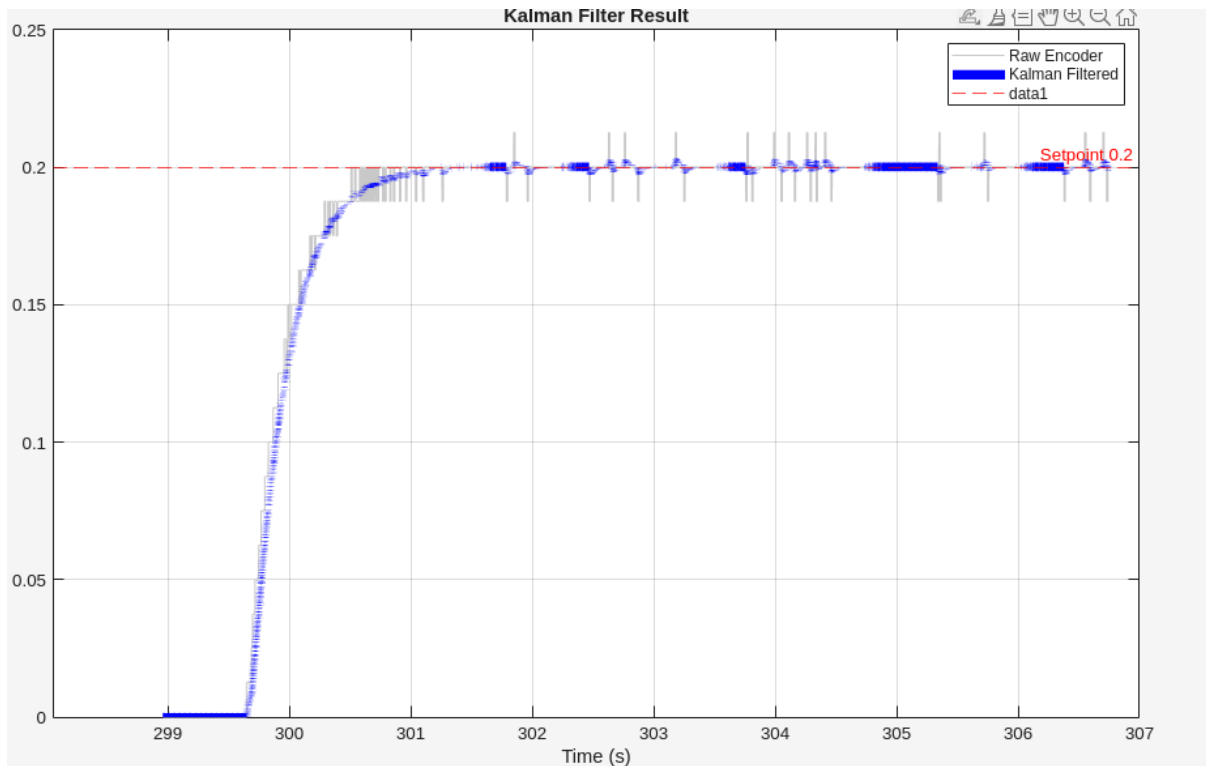


Figure 7-2 $R=0.1$ 을 적용한 결과이다. 측정 노이즈 공분산을 증가시킴에 따라 측정값의 순간적인 변동이 감소하였으며, 보다 부드러운 속도 추정 결과를 얻을 수 있었다(R 을 증가시키면 측정값 보다 예측값을 더 신뢰하게 됨).

7.5 Kalman Filter 적용에 따른 영향

Kalman Filter를 적용한 결과 엔코더 속도 측정값의 순간적인 변동은 감소하였다.

그러나 필터의 특성상 측정값을 평균화하는 과정에서 피드백에 일정한 지연이 발생하였다.

초기 PI Controller는 Raw Encoder 속도를 기준으로 설계되었기 때문에, Kalman Filter를 적용한 이후에는 응답 특성이 다소 변화하였다.

실험 결과 작은 오버슈트와 진동이 발생하는 것을 확인하였다.

이를 보완하기 위해 PI Controller의 게인을 다시 조정하였다.

초기에는 응답 속도를 높이기 위해 PI Gain을 증가시켰으나, Kalman Filter 적용 이후에는 필터 지연을 고려하여 최종적으로 PI Gain을 약 **1.2배 감소**시켜 사용하였다.

7.6 STM32 구현

MATLAB에서 검증한 Kalman Filter를 STM32에 구현하였다.

```
float Kalman_Filter(float z)
{
    P = P + Q;

    float K = P / (P + R);

    x_hat = x_hat + K * (z - x_hat);

    P = (1.0f - K) * P;

    return x_hat;
}
```

속도 계산 과정은 다음과 같다.

```
raw_v=
((float)delta_tick/1632.0f)
*0.2041f
*100.0f;

current_v=Kalman_Filter(raw_v);
```

필터링된 속도 `current_v` 를 PI Controller의 피드백 값으로 사용하였다.

7.7 최종 결과

Kalman Filter를 적용한 후 PI Controller를 재조정하여 최종 속도 제어 성능을 확인하였다.

실험 데이터는 STM32CubeMonitor를 이용하여 기록한 후 CSV 파일로 저장하였으며, MATLAB을 이용하여 데이터를 분석하였다.

시간축은 데이터 기록을 시작한 시점을 기준으로 저장되므로, 실제 모터 구동 시작 시점과는 차이가 있다. 본 실험에서는 데이터 기록을 먼저 시작한 후 모터를 구동하였으며, CSV 데이터를 분석한 결과 모터 속도가 증가하기 시작한 시점은 약 299.649 s로 확인되었다.

목표 속도인 0.2 m/s에 처음 도달한 시점은 약 300.509 s였으며, 이를 기준으로 실제 목표 속도 도달 시간은 약 0.86 s였다.

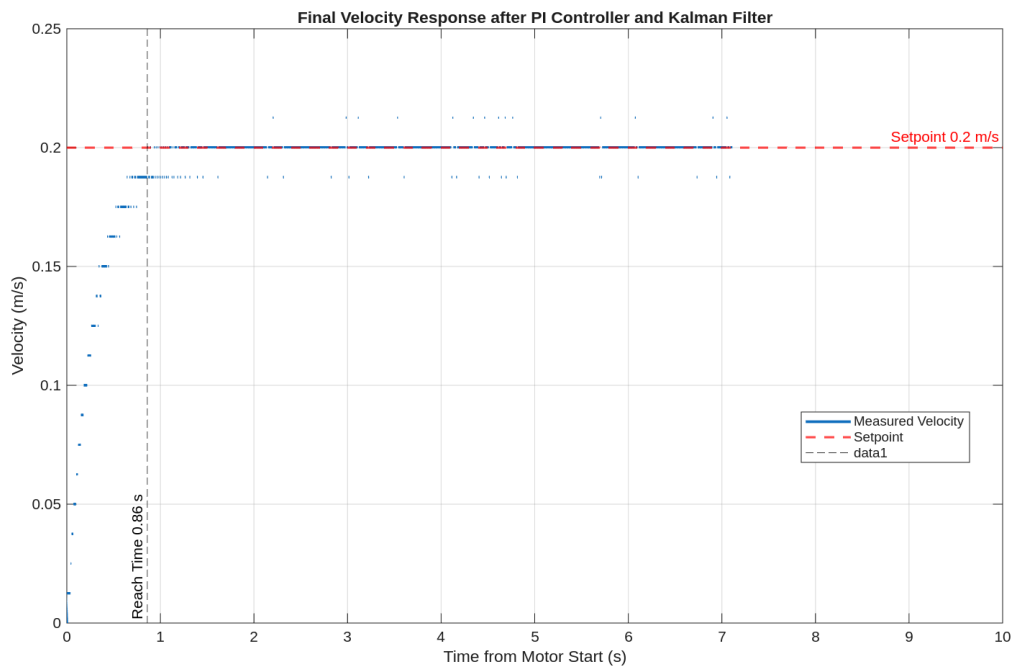
정상상태에서는 평균적으로 목표 속도인 0.2 m/s를 안정적으로 유지하였으며, 최대 속도는 약 0.2126 m/s로 측정되었다. 이는 목표 속도 대비 약 6.3 %의 최대 오버슈트에 해당한다.

또한 정상상태에서 관측되는 작은 속도 변동은 Encoder Tick 기반 속도 계산에서 발생하는 양자화 오차와 측정 노이즈에 의한 것으로 판단하였다. Kalman Filter를 적용함으로써 이러

한 순간적인 속도 변동을 감소시켜 PI Controller에 보다 안정적인 피드백을 제공할 수 있었다.

Figure 7-3에는 CSV 데이터를 MATLAB에서 분석하여 모터 구동 시작 시점을 기준으로 다시 표현한 최종 속도 응답을 나타내었다.

Figure 7-3. Final Velocity Response



CSV 데이터를 MATLAB에서 분석하여 모터 구동 시작 시점을 기준으로 나타낸 최종 속도 응답이다. 모터 구동 후 약 0.86 s 만에 목표 속도 0.2 m/s에 도달하였으며, 이후 정상 상태에서 평균적으로 목표 속도를 안정적으로 유지하였다. 정상상태에서 관측되는 작은 속도 변동은 엔코더의 양자화 오차에 의한 영향이며, Kalman Filter를 통해 측정 노이즈를 감소시켜 보다 안정적인 속도 피드백을 얻을 수 있었다.

8. 실험 결과 및 성능 분석

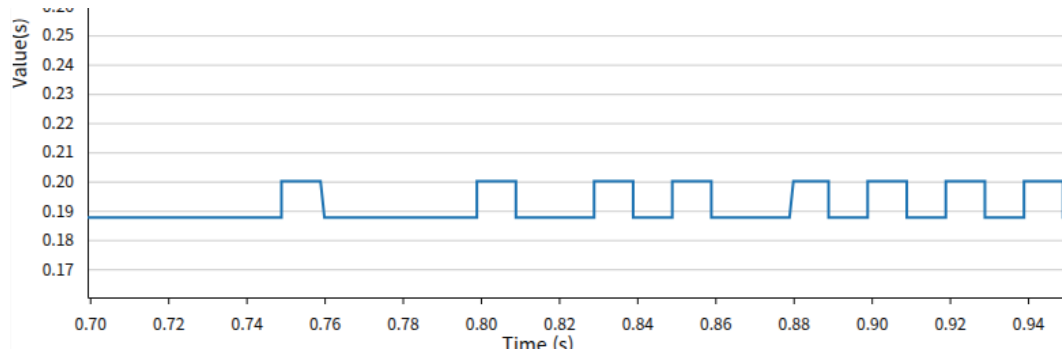
8.1 실험 환경

실험은 STM32F746NG와 JGA25-371 DC 기어모터를 이용하여 수행하였다. 속도 데이터는 STM32CubeMonitor를 이용하여 실시간으로 기록하였으며, 기록된 CSV 데이터를 MATLAB에서 분석하여 제어 성능을 평가하였다.

8.2 PI Controller 적용 결과

PI Controller를 적용한 결과 목표 속도인 0.2 m/s를 평균적으로 추종하는 것을 확인하였다.

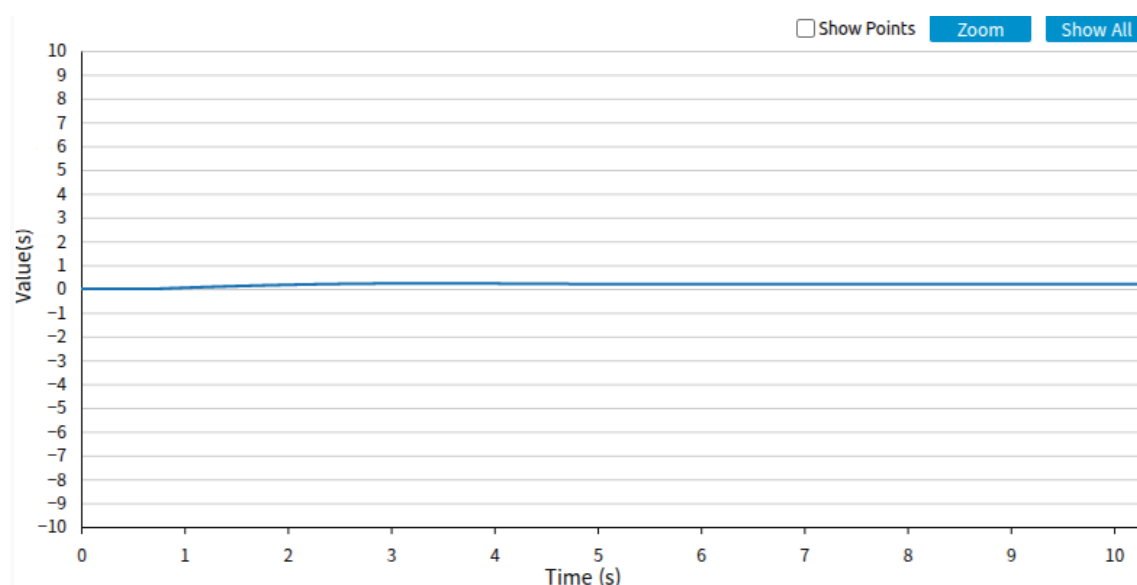
그러나 Encoder Tick 기반 속도 계산 특성으로 인해 정상상태에서는 측정 속도가 일정한 간격으로 변동하는 현상이 발생하였다. 이러한 변동은 실제 모터 속도의 변화라기보다 엔코더 분해능에 의해 발생하는 양자화 오차에 의한 것으로 판단하였다.



8.3 Kalman Filter 적용 결과

Kalman Filter를 적용한 결과 엔코더 측정값의 순간적인 변동이 감소하였으며, PI Controller에 보다 안정적인 속도 피드백을 제공할 수 있었다.

다만 필터의 특성상 측정값을 평균화하는 과정에서 소량의 응답 지연이 발생하였으며, 이를 보완하기 위해 PI Controller의 게인을 다시 조정하였다. 최종적으로 목표 속도에 대한 응답성과 정상상태 안정성을 모두 확보할 수 있었다.



8.4 최종 성능 평가

최종 시스템의 성능을 아래에 정리하였다.

항목	결과
목표 속도	0.200 m/s
목표 속도 도달 시간	약 0.86 s
정상상태 평균 속도	약 0.200 m/s
평균 정상상태 오차	약 0.03 %
최대 속도	약 0.2126 m/s
최대 오버슈트	약 6.3 %
제어 주기	10 ms
속도 추정	1st Order Kalman Filter

최종적으로 목표 속도를 안정적으로 추종하였으며, 평균 속도는 목표값과 거의 일치하였다. 또한 Kalman Filter를 적용하여 엔코더 측정 노이즈를 감소시킴으로써 PI Controller의 입력 신호를 안정화할 수 있었다.

9. 결론

본 프로젝트에서는 무부하 환경에서 속도 제어 성능을 평가하였다. 따라서 다양한 부하 조건에서의 제어 성능은 검증하지 못하였다.

또한 엔코더 기반 속도 측정은 Tick 기반 계산으로 인해 양자화 오차가 발생하였으며, 이를 완화하기 위해 Kalman Filter를 적용하였다. 향후에는 속도 계산 주기와 필터 파라미터를 추가적으로 최적화하여 더욱 안정적인 속도 추정 성능을 확보할 수 있을 것으로 기대된다.

또한 현재 Kalman Filter는 부동소수점(float) 연산으로 구현하였다. 향후에는 고정소수점(Fixed-point) 연산을 적용하여 계산량을 줄이고, MCU 환경에서의 실행 효율을 향상시키는 방법도 검토해 볼 수 있다.

10. 배운 점

본 프로젝트를 통해 PWM 기반 모터 제어, Encoder 속도 측정, System Identification을 이용한 Plant 모델링, 디지털 PI 제어기 설계 및 Kalman Filter 적용 과정을 직접 수행하였다. 특히 연속시간 제어를 디지털 환경에 구현할 때 샘플링 주파수와 제어기 대역폭의 관계를 고려해야 한다는 점을 실험적으로 확인할 수 있었으며, 실제 시스템에서는 모델과 구현 환경의 차이를 고려한 튜닝 과정이 중요함을 경험하였다.